- MQ
  - 优势
    - 应用解耦
      - MQ作为各系统的中间件，使系统的代码最大程度上解耦，后期修改各功能方便
    - 异步提速
      - MQ作为系统间的中间件，可以异步运行，各系统间隔离，提升运行速度
    - 削峰填谷
      - MQ作为中间件，缓冲大量请求，子系统后续进行拉取
  - 劣势
    - 系统可用性降低
      - 分布式系统引入的外部依赖变多(增加了MQ系统)，系统稳定性变差，一旦MQ宕机，会对业务造成影响
    - 系统复杂度提高
      - 异步调用MQ需要保证消息没有被重复消费，消息没有丢失，消息传递的顺序不紊乱
    - 一致性问题
      - MQ的消息在后续不同系统中处理结果不一致的情况
  - 高级特性
    - 消息的可靠投递
      - confirm
        - ConnectionFactory中开启publisher-confirms="true"
        - rabbitTemplate中定义ConfirmCallBack回调函数
          - rabbitTemplate.setConfirmCallback(new RabbitTemplate.ConfirmCallback){
@Override
public void confirm(){}
}
      - return
        - ConnectionFactory中开启publisher-return="true"
        - 定义ReturnCallBack 同上
        - 设置Exchange处理消息的模式
          - 如果消息没有路由到Queue，则丢弃消息(默认)
          - 如果消息没有路由到Queue，返回消息给发送方ReturnCallBack
    - Consumer Ack
      - listener-container中设置属性acknowledge="manual"
      - 让监听器类实现ChannelAwareMessageListener接口
      - 如果消息处理成功，则调用channel的basicAck()签收
      - 如果消息处理失败，则调用channel的basicNack()拒绝签收，broker重新发送给consumer
    - 消费端限流
      - 确保ack机制为手动确认
      - listener-container配置属性perfetch = n，表示消费端每次从mq拉取n条消息来消费，直到手动确认消费完毕后，才会继续拉取下一批消息
    - TTL
      - 统一过期：
rabbit:queue中设置queue-arguments
<entry key="x-message-ttl" value="10000" value-type="java.lang.Integer" />
      - 消息单独过期：
使用MessagePostProcessor对象的postProcessMessage方法处理Message对象，
message.getMessageProperties().setExpiration("5000")
    - 死信队列 DLX
      - x-dead-letter-exchange
      - x-dead-letter-routing-key
      - 消息成为死信的三种情况
        - 队列消息长度到达限制
        - 消费者拒接消息，并且不重回队列
        - 原队列存在消息过期设置，过期未被消费
    - 延迟队列(TTL+DLX)
    - 消息补偿
      - 延时发送一个消息，增加一个回调服务进行数据对比来检漏，并增加一个定时服务定期检漏以防延时消息也没发送成功
    - 消息幂等性保障
      - 消费多条同样的消息结果应为消费一次(比如重复扣费问题)
      - 解决方法(乐观锁)
        - 表中添加version字段
        - 执行更新操作前查询version
        - 执行更新语句，以查到的version作为条件，比如
UPDATE XXX SET VERSION=VERSION+1
WHERE VERSION=1
        - 如果执行更新时有其他人先更新了，则当前操作就不会执行了
    - 集群
      - 负载均衡HAProxy